DURAND Geoffrey
Polytech Sorbonne - EI4

Polytech OS User – TP partie 1

1 – Etape 1 : Création du programme interactif

Voici le résultat obtenu depuis mon terminal :

```
[MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$ make
 gcc -Wall -o biceps biceps.c -lreadline
[MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$ ./biceps
[Geoffrey2bret@MacBook-Pro-de-Geoffrey.local$ test de commande
 test de commande
[Geoffrey2bret@MacBook-Pro-de-Geoffrey.local$
 MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$
```

L'application récupère d'abord les informations du système et de l'utilisateur pour formater la chaîne du prompt. Ensuite, elle entre dans une boucle infinie où la fonction readline bloque l'exécution en attendant une saisie au clavier. Une fois la touche Entrée pressée, la chaîne est affichée à l'écran, stockée dans l'historique, puis la mémoire est libérée avant de relancer un tour de boucle.

2 – Etape 2 : Analyse de la commande

Le code introduit les variables globales `Mots` et `NMots` pour stocker les éléments de la saisie. La fonction `copyString` alloue dynamiquement de la mémoire pour dupliquer une chaîne de caractères. La fonction `analyseCom(char *b)` est appelée à chaque saisie validée : elle commence par nettoyer la mémoire du précédent appel , puis découpe la nouvelle chaîne en utilisant `strsep` avec comme délimiteurs l'espace, la tabulation et le saut de ligne. Chaque mot non vide est copié via `copyString` dans le tableau `Mots`. Dans le programme principal, ces mots sont ensuite itérés pour afficher le nom de la commande et ses paramètres respectifs.

```
[MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$ make
 gcc -Wall -o biceps biceps.c -lreadline
[MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$ ./biceps
[Geoffrey2bret@MacBook-Pro-de-Geoffrey.local$ ls    -l /tmp
 commande : ls
 parametre 1 : -l
 parametre 2 : /tmp
[Geoffrey2bret@MacBook-Pro-de-Geoffrey.local$
 MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$
```

L'analyse de la commande fonctionne parfaitement : le découpage ignore bien les espaces multiples et sépare correctement la commande de ses paramètres.

3 – Etape 3 : Exécution des commandes

3.1 Mise en place des commandes internes

```
[MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$ Make                          ]
gcc -Wall -o biceps biceps.c -lreadline
[MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$ ./biceps                      ]
[Geoffrey2bret@MacBook-Pro-de-Geoffrey.local$ toto                        ]
la commande 'toto' n'est pas reconnue pour le moment.
[Geoffrey2bret@MacBook-Pro-de-Geoffrey.local$ exit                        ]
fermeture du programme...
[MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$ _                             ]
```

Le code repose sur la création d'une structure CommandeInterne qui permet d'associer un nom de commande (comme "exit") à un pointeur vers une fonction C spécifique. On initialise ensuite un tableau global TabComInt limité à 10 emplacements pour répertorier ces commandes. La fonction ajouteCom insère les commandes dans ce tableau , tandis que majComInt est appelée dès le démarrage du programme pour peupler le tableau avec la commande "exit". Lorsqu'une phrase est saisie dans le prompt, elle est d'abord découpée par analyseCom. Ensuite, la fonction execComInt prend ce découpage, compare le tout premier mot avec ceux du tableau TabComInt, et si elle trouve une correspondance, elle exécute la fonction C pointée et renvoie 1. Si aucune correspondance n'est trouvée, elle renvoie 0, ce qui permet à la boucle principale d'afficher un message d'erreur temporaire. La fonction Sortie, quant à elle, se charge de libérer la mémoire allouée par le découpage des mots avant de fermer proprement le processus avec exit(0).

3.2 Gestion des commandes externes

```
MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$ make trace
gcc -Wall -DTRACE -o biceps biceps.c -lreadline
MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$ ./biceps
Geoffrey2bret@MacBook-Pro-de-Geoffrey.local$ ls -l
[trace] processus parent (pid: 1328) attend l'enfant (pid: 1330)
[trace] processus enfant (pid: 1330) va executer : ls
total 64
-rw-r--r--  1 Geoffrey2bret  staff    203 17 mar 14:52 Makefile
-rw-r--r--  1 Geoffrey2bret  staff      0 16 mar 14:35 README.md
-rwxr-xr-x  1 Geoffrey2bret  staff  50976 17 mar 14:54 biceps
-rw-r--r--  1 Geoffrey2bret  staff   4972 17 mar 14:52 biceps.c
[trace] processus enfant termine
Geoffrey2bret@MacBook-Pro-de-Geoffrey.local$ jecrisnimportequoi
[trace] processus parent (pid: 1328) attend l'enfant (pid: 1391)
[trace] processus enfant (pid: 1391) va executer : jecrisnimportequoi
erreur d'execution de la commande: No such file or directory
[trace] processus enfant termine
Geoffrey2bret@MacBook-Pro-de-Geoffrey.local$ exit
[trace] execution de la commande interne : exit
fermeture du programme...
```

Le code intègre maintenant la fonction execComExt qui utilise fork() pour créer un clone du processus principal. Dans ce clone (le processus enfant), la fonction execvp remplace le code du programme par celui de la commande système demandée (comme ls ou pwd), en s'appuyant sur notre tableau de mots qui se termine obligatoirement par NULL. Pendant que l'enfant travaille, le processus parent attend passivement sa complétion grâce à waitpid() pour ne pas afficher le prompt trop tôt. Dans la boucle main, si la commande n'est pas reconnue comme interne (c'est-à-dire que execComInt retourne 0), le relais est automatiquement passé à execComExt pour interroger le système. Des blocs #ifdef TRACE ont été insérés pour afficher les étapes de ce mécanisme uniquement si le programme est compilé avec la directive adéquate.

4 – Etape 4 : biceps version 1

4.1 Des commandes internes indispensables

```
MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$ Make
gcc -Wall -o biceps biceps.c -lreadline
MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$ ./biceps
Geoffrey2bret@MacBook-Pro-de-Geoffrey.local [/Users/Geoffrey2bret/Desktop/EI4/OS/TP1]$ pwd
/Users/Geoffrey2bret/Desktop/EI4/OS/TP1
Geoffrey2bret@MacBook-Pro-de-Geoffrey.local [/Users/Geoffrey2bret/Desktop/EI4/OS/TP1]$ cd /tmp
Geoffrey2bret@MacBook-Pro-de-Geoffrey.local [/private/tmp]$ vers
biceps version 1.0
```

Ce code conserve toute la logique précédente mais ajoute le support des commandes exécutables du système (comme ls, cat, pwd, etc.). Quand l'utilisateur tape une commande, le programme main tente d'abord de l'exécuter via execComInt. Si cette fonction renvoie 0 (ce qui signifie que ce n'est pas "exit" ou une autre commande interne enregistrée), le relais est passé à execComExt. Cette nouvelle fonction utilise fork() pour cloner le processus principal. Le processus parent (notre shell) utilise waitpid() pour se mettre en pause et attendre que la tâche se termine. Pendant ce temps, le processus enfant utilise execvp() pour remplacer son propre code par celui du programme demandé par l'utilisateur (par exemple la commande ls).

4.2 Des commandes externes séquentielles

```
MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$ make
gcc -Wall -o biceps biceps.c -lreadline
MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$ ./biceps
Geoffrey2bret@macbook-pro-de-geoffrey.home [/Users/Geoffrey2bret/Desktop/EI4/OS/TP1]$ ls -l ; vers ; pwd ; echo Il fait beau en ce moment
total 72
-rw-r--r--  1 Geoffrey2bret  staff    203 17 mar 14:52 Makefile
-rw-r--r--  1 Geoffrey2bret  staff      0 16 mar 14:35 README.md
-rwxr-xr-x  1 Geoffrey2bret  staff  51272 19 mar 21:42 biceps
-rw-r--r--  1 Geoffrey2bret  staff   9396 19 mar 21:41 biceps.c
biceps version 1.0
/Users/Geoffrey2bret/Desktop/EI4/OS/TP1
Il fait beau en ce moment
Geoffrey2bret@macbook-pro-de-geoffrey.home [/Users/Geoffrey2bret/Desktop/EI4/OS/TP1]$ mkdir test_dossier ; cd test_dossier ; pwd
/Users/Geoffrey2bret/Desktop/EI4/OS/TP1/test_dossier
Geoffrey2bret@macbook-pro-de-geoffrey.home [/Users/Geoffrey2bret/Desktop/EI4/OS/TP1/test_dossier]$
```

Trois nouvelles fonctions C ont été créées (change_dir, print_wd, version) et associées à leurs mots-clés respectifs (cd, pwd, vers) via ajouteCom dans la fonction majComInt. J'ai également fait une légère modification dans la boucle while (1) du main : le prompt est désormais recalculé à chaque tour de boucle avec getcwd pour inclure le dossier courant. Cela permet de voir visuellement que la commande cd a bien fonctionné et changé le répertoire de travail du processus parent.

4.3 Amélioration de l'utilisation de biceps

```
MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$ make
gcc -Wall -o biceps biceps.c -lreadline
MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$ ./biceps
Geoffrey2bret@MacBook-Pro-de-Geoffrey.local [/Users/Geoffrey2bret/Desktop/EI4/OS/TP1]$ls -l
total 72
-rw-r--r--  1 Geoffrey2bret  staff    203 17 mar 14:52 Makefile
-rw-r--r--  1 Geoffrey2bret  staff      0 16 mar 14:35 README.md
-rwxr-xr-x  1 Geoffrey2bret  staff  51560 20 mar 08:52 biceps
-rw-r--r--  1 Geoffrey2bret  staff  10496 20 mar 08:52 biceps.c
drwxr-xr-x@ 2 Geoffrey2bret  staff     64 19 mar 22:21 test_dossier
Geoffrey2bret@MacBook-Pro-de-Geoffrey.local [/Users/Geoffrey2bret/Desktop/EI4/OS/TP1]$ ^C^D
sortie correcte du programme biceps.
MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$ ./biceps
Geoffrey2bret@MacBook-Pro-de-Geoffrey.local [/Users/Geoffrey2bret/Desktop/EI4/OS/TP1]$ ls -l
total 72
-rw-r--r--  1 Geoffrey2bret  staff    203 17 mar 14:52 Makefile
-rw-r--r--  1 Geoffrey2bret  staff      0 16 mar 14:35 README.md
-rwxr-xr-x  1 Geoffrey2bret  staff  51560 20 mar 08:52 biceps
-rw-r--r--  1 Geoffrey2bret  staff  10496 20 mar 08:52 biceps.c
drwxr-xr-x@ 2 Geoffrey2bret  staff     64 19 mar 22:21 test_dossier
Geoffrey2bret@MacBook-Pro-de-Geoffrey.local [/Users/Geoffrey2bret/Desktop/EI4/OS/TP1]$
```

La capture d'écran valide les améliorations apportées : le programme ignore correctement le signal d'interruption ctrl-c, se ferme proprement avec le message attendu lors de la saisie de ctrl-d (eof), et conserve l'historique d'une session à l'autre, comme le prouve le rappel réussi de la commande ls -l après le redémarrage.

4.4 Restructuration du code

```
MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$ make
gcc -Wall -c biceps.c
gcc -Wall -c gescom.c
gcc -Wall -o biceps biceps.o gescom.o -lreadline
MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$ ./biceps
Geoffrey2bret@MacBook-Pro-de-Geoffrey.local [/Users/Geoffrey2bret/Desktop/EI4/OS/TP1]$ nm gescom.o
0000000000000608 b _Mots
0000000000000610 b _NMots
0000000000000600 S _NbComInt
00000000000000a0 C _TabComInt
                 U ___stderrp
0000000000000140 T _ajouteCom
0000000000000080 T _analyseCom
00000000000002b0 T _execComExt
00000000000001e0 T _execComInt
                 U _execvp
                 U _exit
                 U _fork
                 U _fprintf
                 U _free
0000000000000000 T _libererMots
                 U _malloc
                 U _perror
                 U _printf
                 U _strcmp
                 U _strdup
                 U _strlen
                 U _strsep
0000000000000340 T _traiterSequence
                 U _waitpid
```

La compilation modulaire a généré avec succès les fichiers objets distincts. L'analyse via la commande système nm confirme que la librairie gescom exporte correctement ses fonctions (visibles avec le symbole 'T' pour la section text) tout en maintenant le reste encapsulé.

5 – Etape 5 : biceps version 1.x

5.1 – version 1.1 : gérer les pipes (|)

```
MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$ make clean
rm -f biceps *.o
MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$ make
gcc -Wall -c biceps.c
gcc -Wall -c gescom.c
gcc -Wall -o biceps biceps.o gescom.o -lreadline
MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$ ./biceps
Geoffrey2bret@MacBook-Pro-de-Geoffrey.local [/Users/Geoffrey2bret/Desktop/EI4/OS/TP1]$ ls -l | wc -l
      10
Geoffrey2bret@MacBook-Pro-de-Geoffrey.local [/Users/Geoffrey2bret/Desktop/EI4/OS/TP1]$ pwd ; ls | grep ".c"
/Users/Geoffrey2bret/Desktop/EI4/OS/TP1
Geoffrey2bret@MacBook-Pro-de-Geoffrey.local [/Users/Geoffrey2bret/Desktop/EI4/OS/TP1]$ exit
sortie correcte du programme biceps.
MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$
```

L'exécution valide la bonne implémentation des tubes de communication (pipes). La commande ls -l | wc -l retourne bien le nombre de lignes attendu (10), confirmant que la sortie standard du premier processus fils est correctement redirigée vers l'entrée standard du second via les appels système pipe et dup2. L'enchaînement mixte avec le point-virgule (pwd ; ls | grep ".c") ne plante pas et respecte l'ordre d'exécution.l est à noter que le grep ne retourne aucun fichier ici car notre interpréteur basique ne retire pas encore les guillemets autour des chaînes de caractères lors de l'analyse. La commande grep cherche donc littéralement la chaîne avec les guillemets.

5.2 – version 1.2 : ajouter la gestion des redirections (<<, <, >, >>, 2>, 2>>)

```
MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$ make
gcc -Wall -c biceps.c
gcc -Wall -c gescom.c
gcc -Wall -o biceps biceps.o gescom.o -lreadline
MacBook-Pro-de-Geoffrey:TP1 Geoffrey2bret$ ./biceps
Geoffrey2bret@MacBook-Pro-de-Geoffrey.local [/Users/Geoffrey2bret/Desktop/EI4/OS/TP1]$ echo Je valide le TP 1 > test.txt
Geoffrey2bret@MacBook-Pro-de-Geoffrey.local [/Users/Geoffrey2bret/Desktop/EI4/OS/TP1]$ cat < test.txt
Je valide le TP 1
Geoffrey2bret@MacBook-Pro-de-Geoffrey.local [/Users/Geoffrey2bret/Desktop/EI4/OS/TP1]$ echo C pour un test  >> test.txt
Geoffrey2bret@MacBook-Pro-de-Geoffrey.local [/Users/Geoffrey2bret/Desktop/EI4/OS/TP1]$ cat test.txt
Je valide le TP 1
C pour un test
Geoffrey2bret@MacBook-Pro-de-Geoffrey.local [/Users/Geoffrey2bret/Desktop/EI4/OS/TP1]$ ls fichier_fantome 2> erreur.log
Geoffrey2bret@MacBook-Pro-de-Geoffrey.local [/Users/Geoffrey2bret/Desktop/EI4/OS/TP1]$
```

L'exécution démontre le fonctionnement valide des redirections de flux POSIX. La création et l'écrasement de fichier (>), la lecture depuis un fichier (<), l'ajout en fin de fichier (>>) et la redirection spécifique de la sortie d'erreur (2>) s'opèrent correctement. Le terminal principal reste propre lors des erreurs, ce qui confirme que l'utilisation combinée des appels système open, dup2 et close au sein des processus enfants gère et isole parfaitement les flux.